

Proving Liveness of Load-Store Queue

Follow the *relational ranking* approach of McMillan, *Toward Liveness Proofs at Scale*, CAV 2024 (Rules 6 and 8). Fix e to be an arbitrary entry of the load queue, at slot K .

There needs to be some sort of statement that infinitely often the LSQ accepts requests.

1 In queue entails will be sent

Rule 6, instantiations:

- p : e is in the queue — `load_pending(K)`
- q : e is answered in the queue — `load_rdy(K)`
- r : The memory system responds to an entry before e
- Liveness invariant ϕ : `true`. It isn't needed.
- Ranking relation δ : the things ahead of e , in *both* queues, that are not ready — the *set*, not its cardinality.

This yields the following 4 invariants to prove:

-

1.1 The ranking relation

For a pending load slot K , with $\text{off}(X) = X - \text{load_head}$, $\delta(K)$ has one arm per queue:

`loads_not_ready_before(K, L) =`
`load_pending(L) & ~load_rdy(L) & off(L) < off(K)`

`stores_not_ready_before(K, J) =`
`store_pending(J) & mark_offset(J) <= off(K)`

$\delta(K)$ is the pair, ordered componentwise: it is *conserved* when neither arm gains an element and *reduced* when one arm loses one and neither gains. Two arms rather than one because the two queues are indexed by different sorts, `load_idx` and `store_idx`; the paper's Definition 1 admits exactly this — a ranking is an indexed set of relations, “each predicate may be over a different sort”. This is the same split the two summands of a numeric measure would have had, minus the addition.

`mark_offset(J) <= off(K)` is “store J is older than load K ” in pure offset arithmetic (`cross_s`, `cross_l`) — the same idiom `store_elig` and `l_gap` already use, so no new tag reasoning enters. A

pending store is never *ready*; it leaves only by departing, so every older pending store counts, which is what makes the second arm the store queue’s share of the ranking.

Both arms are indexed off a register (`load_pending/store_pending` compare against `load_head/store_head`), so no unrolling over the four offsets is needed and no `while` appears: as relations they are stated once, universally, and the solver instantiates them at the slots it needs. This is the second thing the relational form buys — the numeric version had to be written out four times per arm to be summable.

1.2 Reused relations

The ranking needs three things and two of them already exist.

- `stores_not_ready_before(K, store_head)` is $\sim\text{load_elig}(K)$ for a pending `K`: `load_eli`’s body says either the store queue is empty — so `store_pending(store_head)` is false — or `store_head`’s mark sits strictly above `off(K)`. Worth stating once, since statements 1 and 3 both lean on it:

```
invariant [elig_zero] load_pending(K) ->
  (load_eli(K) <-> ~stores_not_ready_before(K, store_head))
```

It is also the bridge that lets the store arm inherit the existing tag-order facts (`cross_s`, `cross_l`) rather than needing new ones. Note what is *not* needed: the numeric version had to turn “the counted set is non-empty” into “`store_head` is in it”, which is `m_mono`. A relation is reduced by exhibiting a witness, and statement 3 names `st_head` as the witness in its own hypothesis, so `m_mono` no longer has to carry that step.

- `store_pending` and `load_pending` are the window guards in the two arms; `load_pending_pre` is just the latter’s body restated over the pointer wires.
- `store_eli` is *not* reusable for the load arm, though it reads close to it. Its body lets a load ahead be sent-but-unanswered provided the addresses differ; the ranking has to include that load, because it is not ready and the cache still owes it. So `loads_not_ready_before` is the one genuinely new relation here.

Why this ranking and not “the entries before `e`”:

event	effect on <code>delta(K)</code>
response captured at an offset < <code>off(K)</code>	loses <code>cache_to_lsq_resp_idx</code>
response captured at <code>K</code>	<code>q</code> reached
response captured after <code>K</code>	conserved
store departs (<code>st_go</code>) and was older than <code>K</code>	loses <code>st_head</code>
store arrives	conserved — its mark lands at <code>off(ld_tail)</code> , younger than <code>K</code>
load arrives	conserved — it lands at the tail
head load retires (<code>ld_ans</code>)	conserved — the retired head was ready, so it was never in <code>delta</code>
port grant to a load	conserved

The plain “entries before `e`” relation loses an element only on a retire, which is not what the cache’s justice condition delivers; this one loses one on exactly the two events the cache is fair about.

Note the retire row. Both arms are stated in *offsets* from `load_head`, and a retire shifts `load_head`

and `mark_offset` together, so every membership is preserved across the shift. As a set of *slots* the extension does not move at all, which is what “conserved” has to mean.

2 The four statements

1. **Enabledness at the bottom of the ranking.** In the rule this premise is $\phi \rightarrow \text{En}(\tau)$, a *single-state* formula — note the absence of any `_pre` — so it is a plain *invariant* in implementation next to `outst` and `pres_l`, evaluated in one state rather than paired across an edge, and it needs no sample and no new wire:

```
load_pending(K) & load_elig(K)
  & (forall L. ~loads_not_ready_before(K, L))
  & ~load_rdy(K) & ~load_sent(K) & ~req_en_r
  -> <the arbiter presents K on the port>
```

Written with `load_elig` rather than an empty store arm: by `elig_zero` those say the same thing, and `load_elig` is the vocabulary the arbiter’s own guard, `pres_l`, and `outst` are already stated in. What remains, the empty load arm, is the whole content of the statement: every slot ahead of `K` is ready, so none of them satisfies the arbiter’s guard, so `K` wins the priority chain — which resolves lowest-offset-last, hence lowest-offset-wins.

The bottom of the ranking is needed here. Above it the helpful transition is not `K`’s at all but whatever removes an element ahead of it, and `K`’s own request is genuinely blocked: `load_elig(K)` fails while an older store is pending, and a lower-offset unready load outranks `K`. Justice on `creq_ready` then takes `K` off the port.

2. **The ranking is conserved.** This is conserves `delta`, i.e. forall `x`. $\text{delta}'(x) \rightarrow \text{delta}(x)$, one conjunct per arm:

```
load_pending_pre(K)
  -> load_rdy(K)
  | ( (loads_not_ready_before(K, L) -> loads_not_ready_before_pre(K, L))
    & (stores_not_ready_before(K, J) -> stores_not_ready_before_pre(K, J)) )
```

No comparison and no arithmetic on the ranking: the unprimed relations are read post-edge and the `_pre` ones are the pre-edge samples. The `load_rdy` disjunct absorbs the retire case, where `off(K)` wraps to 7. It is sound there: the retiring slot cannot be re-enqueued on the same edge, because `cpu_ready_r` is low whenever the queue is already at capacity (`q_l`, `q_rdy`), so `load_rdy(K)` still holds post-edge.

3. **The helpful step reduces it.** reduces `delta` is exists `x`. $\text{delta}(x) \& \sim \text{delta}'(x)$, and in both halves the witness is already available, so it can be named rather than quantified. Two halves, because `cache_to_lsq_resp_valid` on its own reduces nothing — the response may be for a slot after `K`, or match no sent entry at all; nothing in `lsq_body` constrains `resp_idx`, the whole channel is a free input.

```
load_pending_pre(K) & ld_capture_pre
  & (resp_idx_pre - lh_pre) < (K - lh_pre)
  -> loads_not_ready_before_pre(K, resp_idx_pre)
  & ~loads_not_ready_before(K, resp_idx_pre)

load_pending_pre(K) & st_go_pre & stores_not_ready_before_pre(K, sh_pre)
  -> ~stores_not_ready_before(K, sh_pre)
```

The load half’s witness is `resp_idx_pre`, the response index sampled at the start of the edge: whichever of `ld_capture`’s four per-offset guards fired, that slot is exactly the one the sample recorded, so the existential witness is already in hand. `ld_capture` itself is new only as the disjunction of those four guards, each already the datapath’s own capture condition, restated. The store half’s hypothesis, `stores_not_ready_before_pre(K, sh_pre)`, is the pre-edge form of `~load_elig(K)` (`elig_zero` again) and is simultaneously the witness the conclusion discharges. It is carried as a sample because `load_elig` reads `mark_offset`, which by the time this invariant is checked a same-edge retire may already have shifted.

4. **Justice.** An assumption on the environment, with nothing to prove: the cache accepts the port (`cache_to_lsq_creq_ready`) infinitely often, and answers every load it has taken. Documented as a comment where the freeness of the cache channel is already described. This is stronger than “the cache responds infinitely often”: the capture guard tests `resp_idx` and the echoed address, so bare `resp_valid` liveness would leave a sent load unanswered forever.

3 Soundness

`delta` is a relation over `load_idx` and `store_idx`, which `basis` interprets as `bv[3]`. Inclusion on subsets of a finite set is well-founded, so no order needs axiomatising and no induction axiom needs instantiating — which is the property the paper is buying, and the reason it replaces the numeric rank.

Finiteness, premises F1/F2 of Rule 5, is therefore free: the paper needs that rule only for rankings over infinite sorts, and it discharges it by showing each transition adds boundedly many elements. Here the sorts are finite by construction, so `delta` is finite in every state whatever it contains. Even taking the obligation at face value it is immediate: one edge enqueues at most one entry, a load or a store, never both — the accept step is guarded by `lsq_to_cpu_ready` and branches on `cpu_to_lsq_req_wen` — so `delta` can gain at most that one entry — and the offset guards exclude it, since a new entry lands at the tail, younger than the pending `K`. That is the row-by-row content of the event table above, and it is what makes statement 2 provable rather than merely true.

The argument the four statements feed is Rule 6, with the two halves of statement 3 standing for the two justice conditions of Rule 8 under the priority scheduler the paper describes: `psi_store` is “an older store is still pending”, i.e. `stores_not_ready_before(K, st_head)`, and `psi_load` is its negation. Both are stable in the paper’s sense — a pending store leaves only by departing (`st_go`), and once no older store is pending none can appear, since a new store’s mark lands at `off(ld_tail)` — and their disjunction is `true`, so some justice condition is always scheduled. Statement 1 discharges the empty ranking, statement 3 says a non-empty one is left strictly smaller by the scheduled condition, statement 2 says nothing in between undoes that, and statement 4 is what forces the reducing step to actually be taken rather than merely stay possible.

4 Not covered

This proves `load_pending(e) -> eventually load_rdy(e)`. Reaching “the cpu sees the response” additionally needs `load_rdy(e) -> eventually ld_ans` at `e`, which is the “entries before `e`” relation with the retire as the helpful step — a second, much smaller ranking argument on top of this one.